ITA0517 COMPUTER VISION

Experiment 1: Image Reading and Grayscale Conversion

Aim: To read an input image using Python OpenCV and convert it into a grayscale image.

Procedure

1. Import the OpenCV library (cv2) in Python.
2. Specify the file path of the input image.
3. Read the image into memory using the cv2.imread() function.
4. Convert the color image (BGR) to grayscale using cv2.cvtColor() with cv2.COLOR_BGR2GRAY.
5. Display both the original and grayscale images using cv2.imshow().
6. Hold the display windows with cv2.waitKey(0) and clean up resources using cv2.destroyAllWindows().

Code

```
import cv2

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

cv2.imshow("Original Image", img)

cv2.imshow("Grayscale Image", gray)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Input



Output



Result: The input color image was successfully read and converted into a grayscale image using Python and OpenCV.

Experiment 2: Image Blurring using Gaussian Blur

Aim: To read an input image using Python OpenCV and apply Gaussian Blur to smooth the image.

Procedure

1. Import the required cv2 library.
2. Define the path location of the image file.
3. Read the image from the specified path using cv2.imread().
4. Apply the Gaussian Blur filter using cv2.GaussianBlur() with a designated kernel size (e.g., 7×7).
5. Display the original image and the blurred output image using cv2.imshow().
6. Close all active windows on user input using cv2.destroyAllWindows().

Code

```
import cv2

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")

blur = cv2.GaussianBlur(img, (7, 7), 0)

cv2.imshow("Original Image", img)

cv2.imshow("Blurred Image", blur)

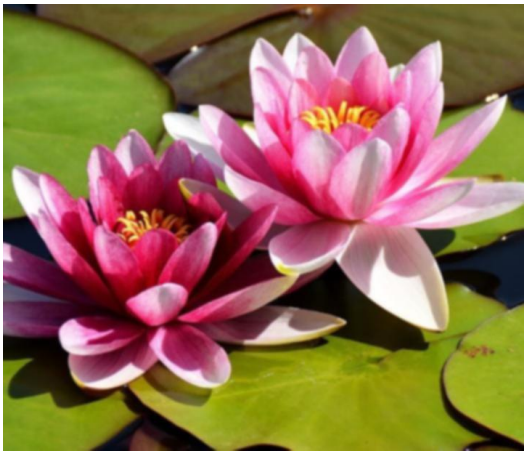
cv2.waitKey(0)

cv2.destroyAllWindows()
```

Input



Output



Result: The input image was successfully loaded and converted into a smoothed/blurred image using the Gaussian Blur function.

Experiment 3: Edge Detection using Canny Function

Aim: To read an input image using Python OpenCV and extract its structural outline using the Canny edge detection function.

Procedure

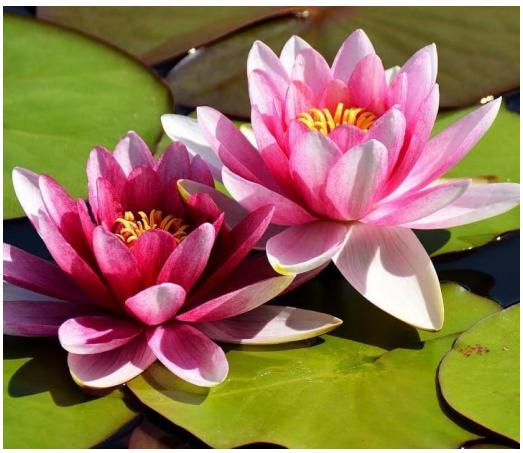
1. Import the OpenCV package (cv2).
2. Provide the absolute file path of the source image.
3. Read the input image using cv2.imread().
4. Convert the color image to grayscale to simplify gradient processing.

5. Pass the grayscale image through cv2.Canny() with lower and upper hysteresis thresholds.
6. Display the detected outline/edge map and destroy windows after key press.

Code

```
import cv2
img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 100, 200)
cv2.imshow("Original Image", img)
cv2.imshow("Canny Edges", edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The image outline and prominent edges were successfully extracted and displayed using the Canny edge detection algorithm.

Experiment 4: Histogram Equalization

Aim: To perform histogram equalization on an input image using OpenCV and compare it with the original image.

Procedure

1. Import cv2 and numpy modules.
2. Specify the image path and read the image directly in grayscale mode (cv2.imread(path, 0)).
3. Apply cv2.equalizeHist() on the grayscale image array to adjust global contrast.
4. Concatenate the original and equalized images horizontally using np.hstack().
5. Display the side-by-side comparison window using cv2.imshow().
6. Release window resources upon pressing any key.

Code

```
import cv2

import numpy as np

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg", 0)

equ = cv2.equalizeHist(img)

res = np.hstack((img, equ))

cv2.imshow("Original vs Equalized", res)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Input



Output



Result: Histogram equalization was successfully applied to enhance the contrast of the original image, and a comparative output was generated.

Experiment 5: Color Level Histogram Analysis

Aim: To write a Python function that analyzes and plots the histogram of an input image across its individual color channels using OpenCV.

Procedure

1. Import cv2 and matplotlib.pyplot packages.
2. Create a custom Python function that accepts the image path as an argument.
3. Read the image and define a list corresponding to BGR color channels.
4. Loop through each color channel and compute its intensity distribution using cv2.calcHist().
5. Plot the calculated histogram curves using Matplotlib with corresponding colors (b, g, r).
6. Display the plot window with appropriate axis labels and title.

Code

```
import cv2

import matplotlib.pyplot as plt

def analyze_histogram(image_path):

    img = cv2.imread(image_path)

    colors = ('b', 'g', 'r')

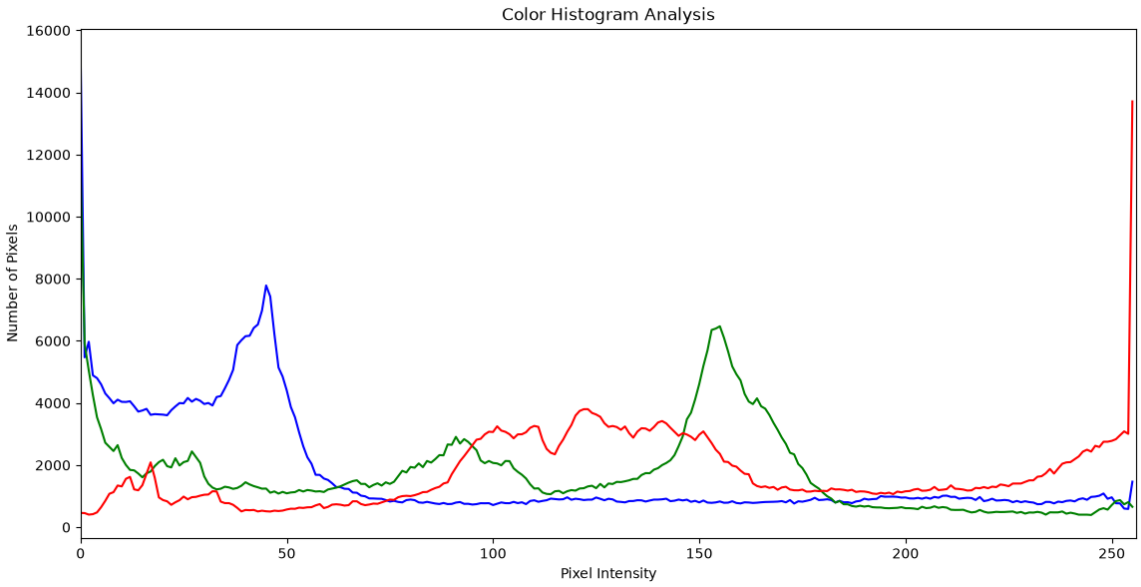
    for i, col in enumerate(colors):
```

```
        hist = cv2.calcHist([img], [i], None, [256], [0, 256])
        plt.plot(hist, color=col)
        plt.xlim([0, 256])
    plt.title("Color Histogram Analysis")
    plt.xlabel("Pixel Intensity")
    plt.ylabel("Number of Pixels")
    plt.show()
analyze_histogram(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
```

Input



Output



Result: The Python function for analyzing color level histograms was successfully implemented and executed using OpenCV.